STUDENT ATTENDANCE TRACKER

Final Requirements, Architecture & Implementation Plan

A learning-focused, multi-user web application for semester-based attendance, timetable management, Duty Leave, imports/exports, and controlled data archival.

Area	Decision
Frontend	React + TypeScript + Vite
Backend	Node.js + Express
Database	MongoDB + Mongoose
Planned cache compatibility	Redis adapter in Version 2; cache interface designed in Version 1
Primary design goal	Correct attendance history, secure student data isolation, and incremental learning

Status: Requirements planning only. No implementation has started.

Prepared from the decisions finalized during project planning. Treat this document as the current source of truth; any future change should update the requirements before implementation.

Document map

Section	Topic
1	Project overview and principles
2	Application pages and navigation
3	Functional modules
4	Attendance rules and calculations
5	Timetable and class occurrence design
6	Attendance import format and workflow
7	Duty Leave model and reconciliation
8	MongoDB data model
9	Indexes and query patterns
10	API outline
11	Authentication, security, and validation
12	Semester archival and downloads
13	Redis-ready architecture
14	Version roadmap and seven-day stretch plan
15	Testing and acceptance criteria
16	Project structure and implementation instructions
17	Decisions, assumptions, and out-of-scope items

Important: completed semesters are retained in cloud object storage for download only. The application must not load archived detailed data back into MongoDB or query it for historical dashboards, calculators, or the query console.

1. Project overview and principles

The Student Attendance Tracker is a multi-user web application. Each student creates an account, maintains the current semester timetable, records attendance per individual class period, tracks Duty Leave, views current-semester insights, imports attendance from a defined spreadsheet template, and exports or archives their own data.

1.1 Final technology choices

Layer	Technology / decision	Reason
Frontend	React + TypeScript + Vite	Component-based UI; TypeScript adds type safety while learning React.
Routing / UI	React Router, plain CSS, reusable components	Separate pages with a consistent application shell.
Backend	Node.js + Express; CommonJS initially	Straightforward API setup and learning path.
Database	MongoDB + Mongoose	Document storage and schema validation.
File import/export	CSV and XLSX support	Template-based attendance upload and personal downloads.
Caching	Cache interface in V1; Redis adapter in V2	Avoid a later architectural rewrite.
Deployment	Local development first; container/cloud later	Keep the first milestone focused on correctness.

1.2 Non-negotiable principles

- Every student can access only their own records. The backend derives ownerId from authentication and never trusts a client-provided ownerId.
- MongoDB is the source of truth for the active semester. Redis, when introduced, is only a cache.
- Every class period is independent. Consecutive lectures and multi-period practicals are not merged into one attendance record.
- Timetable changes must never rewrite past attendance. Historical class details are preserved through snapshots or versioned schedule data.
- Overlapping timetable entries are allowed. Warn about overlaps; do not prevent saving.
- Attendance import must preview and validate before committing. No silent overwrites or guessed matches.
- Completed semesters are archived to cloud storage and removed from live detailed collections only after successful verification. Archived data is download-only.
- Build in phases. Core data correctness and security take priority over visual polish and optional advanced features.

2. Application pages and navigation

Page / route	Main responsibilities	Phase
/register, /login	Self-registration, sign-in, logout, authenticated session/token handling.	V1 / P0
/dashboard	Current-semester summary, today's classes, pending marking, below-target alerts, quick actions.	V1 / P0
/semesters	Create/select current semester, dates, complete/archive workflow; show archived entries as download-only.	V1 / P0
/semesters/:id	Current semester details, status, date range and subject overview.	V1 / P0
/subjects	Create/edit/archive subjects, codes, attendance targets and summaries.	V1 / P0
/timetable	Weekly recurring schedule, teacher, room, L/P type, period separation, effective-date changes.	V1 / P0
/attendance	Date-based period list, status marking, overlap confirmation, history and correction.	V1 / P0
/calendar	Holidays, cancelled classes, exceptions, extra classes, date navigation.	V1 / P0
/duty-leave	Applications, eligible-period matching, approval status, credits and reconciliation.	V1 / P0 basic
/analytics	Attendance trends, target calculator, absence allowance, subject summaries.	V1 / P1
/query-console	Restricted read-only MongoDB-style query interface for active-semester data only.	V1 / P1
/data	Template download, import preview/commit, validation report, CSV export.	V1 / P1
/planner	Exams, assignments, study tasks and due dates.	V2 / P2
/settings	Profile and preferences.	V1

Use a persistent sidebar and header with a semester selector. Current-semester pages share the selected semester context. Archived semesters are shown as archived and offer download actions only; selecting one must not switch live pages into archived-data mode.

2.1 Shared layout behavior

- The selected current semester is visible in the header.
- Related operations should use dialogs, side panels, or inline forms instead of creating a separate page for every action.
- The app must distinguish loading, empty, validation-error, permission-denied, and server-error states.
- Mobile responsiveness is desirable, but correctness and desktop workflow come first for the initial milestone.

3. Functional modules

3.1 Authentication and accounts

- Students can self-register, log in, log out, and inspect their own authenticated profile.
- Store password hashes only; never return passwordHash to the client.
- Use secure cookie/session or token handling appropriate to the selected implementation. Apply rate limits to authentication endpoints.
- Every route that reads or changes academic data must be authenticated and owner-scoped.

3.2 Semester and subject management

- Create a semester with name, start date, end date, and status.
- Maintain subjects with name, optional code, semester association, active/archive state, and an attendance target percentage.
- Subject and semester archive/delete operations must not cascade-delete historical records unexpectedly.
- Only the active semester's detailed operational data remains in MongoDB after semester archival policy is applied.

3.3 Timetable management

- A timetable is a recurring weekly schedule for the semester; students enter it once rather than retyping it each week.
- Each entry includes subject, weekday, start/end time, type L/P, teacher name, classroom/lab, effective date range, and schedule version.
- Back-to-back classes remain separate entries, even when subject, teacher, room, and type match.
- A multi-period practical is represented by separate period entries/occurrences. It may be visually grouped, but not merged in storage or attendance.
- Overlapping classes are allowed. Warn on overlap and allow saving.
- Changes can apply to future classes, one occurrence only, or a new schedule version from an effective date.
- Teacher/room/type changes must not rewrite historical attendance snapshots.

3.4 Attendance recording

- Each scheduled period has one of four statuses: PENDING, PRESENT, ABSENT, CANCELLED.
- PENDING is the default UI state when no explicit attendance record exists; do not pre-create all pending documents.
- PRESENT and ABSENT count in the attendance denominator. PENDING and CANCELLED do not.
- A cancelled period must be represented by a cancellation/exception or explicit cancelled status and must not count as an absence.
- Students can browse attendance by date and subject and correct historical records where the application permits it.
- An occurrence identity must distinguish different entries even when they overlap or occur back-to-back.

3.5 Overlapping class confirmation

- When the student marks one class PRESENT, detect other scheduled classes overlapping that class.
- Show a confirmation listing the overlapping classes.
- If confirmed, mark only overlapping PENDING classes ABSENT; do not overwrite classes already PRESENT, ABSENT, or CANCELLED.
- If the student cancels the confirmation, save the selected class as PRESENT but leave the other overlapping classes unchanged.
- Apply the same rule to regular and extra classes. Use a safe, retry-aware backend operation.

3.6 Calendar and exceptions

- Support holidays, cancelled classes, extra classes, and one-off changes.
- A holiday should prevent ordinary scheduled classes from being counted as absences.

- An extra class is a separate occurrence and must not be confused with a recurring timetable entry.
- Define exception precedence clearly when a holiday and a manually created extra class occur on the same date.

3.7 Class notes and assignments

- Students can optionally record topic covered, assignment given, and optional due date for an individual class occurrence.
- Do not require notes when marking attendance.
- Notes belong to a specific occurrence; they must not carry over automatically to the next weekly class.
- Assignment due dates may be surfaced in the Academic Planner in a later phase.

3.8 Dashboard and analytics

- Show today's classes, attendance-marking progress, overall and subject attendance, and below-target alerts.
- Overall attendance is period-weighted: total attended eligible periods divided by total eligible periods. Do not average subject percentages.
- Support date/week/month trends, subject comparisons, and what-if calculators.
- Dashboard calculations operate only on active-semester data. Archived semesters are not queried by the server.

3.9 Read-only query console

- Provide a restricted MongoDB-style query console for active-semester records owned by the logged-in student.
- Allow find(), findOne(), countDocuments(), approved filters, projections, sorting, limits, and a restricted aggregation subset if safely implemented.
- Reject writes, administrative commands, arbitrary JavaScript, unsafe operators, and unrestricted server-side execution.
- Enforce owner scope in the backend regardless of query text; redact sensitive fields and cap execution time/result count.
- Do not query cloud archives. Archived records are available only through the student's file downloads.

3.10 Academic planner and reminders

- Later-phase feature: exams, assignments, study tasks, priorities, due dates, completion state and subject links.
- Reminders are optional and should be added after core attendance and data workflows are reliable.

4. Attendance rules and calculations

4.1 Status and denominator rules

Status	Stored explicitly?	Counts as held/eligible period?	Counts as attended?
PENDING	No document required; inferred if no status exists	No	No
PRESENT	Yes	Yes	Yes
ABSENT	Yes	Yes	No
CANCELLED	Via explicit occurrence/exception status	No	No

Actual attendance percentage = P / (P + A) x 100, where P is present periods and A is absent periods. If P + A = 0, display N/A rather than zero.

4.2 Target calculator

For target T percent, current present periods P, and absent periods A, the minimum additional consecutive periods that must be attended to reach target T (for T < 100) is:

```
N = max(0, ceil((T * (P + A) - 100 * P) / (100 - T)))
```

If T = 100% and A > 0, attending future classes alone cannot restore 100%. If there are no absences, no extra periods are needed.

Maximum additional absences while retaining target T > 0:

```
maxAbsences = floor(100 * P / T - (P + A))
```

If this value is negative, the student is already below target. Clamp/report according to product rules; do not show a negative number as a usable absence allowance.

4.3 Duty Leave adjusted attendance

Keep actual attendance separate from the policy-adjusted percentage. Approved eligible DL credits are a subset of original absent periods and must never double-count an occurrence.

```
Actual % = P / (P + A) * 100
Adjusted % = (P + D) / (P + A) * 100
```

D is the number of approved, eligible, applied DL credits that correspond to originally absent periods. Pending applications are not confirmed attendance; a projected figure may be shown separately and clearly labeled.

4.4 Partial overlap for Duty Leave

Class interval and Duty Leave interval use half-open time ranges [start, end). A class overlaps a DL interval when:

```
classStart < dlEnd && classEnd > dlStart
```

A partial overlap makes the whole class period eligible for review. It does not automatically grant credit: the application must be approved and the period must satisfy the policy. Credit the full period, not a fraction of minutes. A class ending exactly when DL starts does not overlap.

4.5 College block reason

Block Reason means the college has blocked the student for a reason such as a disciplinary incident. Preserve the reason as separate source information and flag it for review. Do not automatically treat a block as a holiday, cancellation, or absence adjustment until the product policy explicitly defines its attendance effect.

4.6 Attendance integrity

- Do not silently turn missing attendance into ABSENT; missing status remains PENDING.
- Do not silently overwrite existing statuses during imports.
- Use stable occurrence identity and idempotent write behavior to prevent duplicate records.

- Preserve an audit trail for corrections and DL credits where feasible.

5. Timetable and occurrence design

5.1 Schedule definition versus occurrence versus attendance

Entity	Meaning	Typical persistence
Timetable entry	Recurring weekday/time rule with subject, type, teacher, room and effective dates.	One record per recurring period/version.
Class occurrence	One specific class on one date; contains stable identity and snapshot details.	Materialize when needed or when a special event requires it.
Attendance record	Explicit PRESENT/ABSENT status and relevant audit metadata.	Create when a status is explicitly recorded.
Class notes	Optional topic/assignment for one occurrence.	Create only when notes exist.
Calendar exception	Holiday, cancellation, one-off change, or extra class.	Create only for exceptions.

5.2 Lazy materialization

Do not generate every future class and every PENDING attendance document for every student. On a date request, compute the expected classes from the applicable timetable version and exceptions, then look up explicit attendance records. If no attendance record exists, render PENDING. Persist explicit statuses, exceptions, extra classes, notes, and any occurrence snapshot needed to preserve historical details.

5.3 Historical consistency

- A timetable edit has an effective date or applies to a single occurrence.
- Past attendance retains the subject, date/time, type, teacher and room that applied to that occurrence.
- A stable occurrenceKey distinguishes timetableEntryId + date (and period identity where required); extra classes use their own stable ID.
- Do not create a unique index on date/time alone because overlaps are allowed.
- Back-to-back periods must remain separate even when all other fields are equal.

6. Attendance import format and workflow

6.1 Required template columns and exact meanings

Column	Meaning / validation
Date	Class date. Required.
Time	Class start/end time or agreed time representation. Required to identify the class.
Type	L = Lecture; P = Practical. Optional; fill from timetable if blank.
Attendance	Status to import. Required; accepted values must be defined and normalized (for example PRESENT/ABSENT, with explicit mapping from source labels).
Name	Teacher name. Optional; if blank, fill from the matching timetable entry.
Block Reason	Optional reason the college blocked the student, e.g. disciplinary action. Preserve separately and flag for review when present.

```
Date | Time | Type | Attendance | Name | Block Reason
```

Matching rule: the student first selects the semester and subject. Match using the selected subject plus Date and Time only. Type, Name, and Block Reason are not matching keys.

6.2 Missing values and conflict handling

- Date, Time and Attendance are required to create/update a status record.
- If Type is blank, infer it from the matching timetable entry.
- If Name is blank, fill the teacher name from the matching timetable entry.
- Block Reason may be blank. If supplied, preserve it; do not infer an attendance effect.
- If the uploaded type or teacher conflicts with the timetable, warn in preview. Do not use those fields to silently redirect the match.
- If multiple classes for the selected subject could match the same date/time, flag ambiguity for user resolution.
- If no timetable entry matches, flag the row for review; do not invent a class. A separately supported extra-class workflow may resolve it.
- Detect duplicate rows and existing records. Never silently overwrite attendance.

6.3 Import workflow

Stage	Behavior
1. Select	Choose current semester and a subject from its timetable/subject list.
2. Upload	Accept .xlsx or .csv using the exact six headers.
3. Parse	Normalize date/time and status values; reject malformed files.
4. Match	Match by selected subject + date + time; use timetable for optional field enrichment.
5. Preview	Show valid rows, inferred values, duplicates, conflicts, and row-level errors.
6. Confirm	Student confirms the proposed changes; no writes occur before confirmation.
7. Commit	Use idempotent writes, report created/updated/skipped/rejected rows, and invalidate relevant caches.

First release scope: templates only. Arbitrary spreadsheet auto-detection and unrestricted column mapping are out of scope. Provide downloadable CSV/XLSX templates and an attendance CSV export.

6.4 Import APIs

```
GET /api/v1/imports/templates/:type
POST /api/v1/imports/preview
POST /api/v1/imports/commit
GET /api/v1/imports/:id
```

7. Duty Leave model and reconciliation

7.1 Application fields

- Event name and reason/reference; optional supporting document reference.
- Start/end date and time; status: PENDING, APPROVED, REJECTED, WITHDRAWN.
- Expected periods, matched periods, approved periods, applied credits, pending/unmatched periods.
- A reconciliation view at semester end before the archive is created.

7.2 Data integrity

- Do not change an original ABSENT record to PRESENT because of DL.
- Store a separate DL credit linked to the application and specific class occurrence.
- Only approved, eligible credits contribute to adjusted attendance.
- Prevent duplicate credits for the same application/occurrence; check that one absence is not credited twice under overlapping applications.
- Evaluate each class period separately, including overlapping timetable classes.

7.3 Collections

Collection	Core fields
dlApplications	ownerId, semesterId, eventName, startAt, endAt, reason, reference, status, createdAt, updatedAt
dlCredits	ownerId, semesterId, dlApplicationId, occurrenceKey, creditStatus, eligibleReason, creditedAt, audit metadata

8. MongoDB data model

Use separate collections and references for growing data. Do not embed an entire semester's attendance in one document. All academic records include ownerId and semesterId where applicable. Mongoose schemas validate fields and enums; backend services enforce ownership and business rules.

users

Field	Type	Purpose / rule
_id	ObjectId	Primary key
name	String	Required display name
email	String	Normalized lowercase; unique index
passwordHash	String	Never exposed to API responses
preferences	Object	Optional UI preferences
createdAt / updatedAt	Date	Audit timestamps

semesters

Field	Type	Purpose / rule
_id	ObjectId	Primary key
ownerId	ObjectId	User reference; always owner-scoped
name	String	Semester display name
startDate / endDate	Date	Semester range
status	Enum	CURRENT, COMPLETED, ARCHIVING, ARCHIVED, ARCHIVE_FAILED
archiveId	ObjectId/String	Reference to archive catalogue when archived

subjects

Field	Type	Purpose / rule
_id	ObjectId	Primary key
ownerId / semesterId	ObjectId	Ownership and semester scope
name / code	String	Code optional; normalized for uniqueness when present
targetPercentage	Number	Validated within product-defined range
active	Boolean	Soft archive flag

timetableEntries

Field	Type	Purpose / rule
_id	ObjectId	Stable entry identity
ownerId / semesterId / subjectId	ObjectId	References
dayOfWeek	Number	Defined weekday convention
startMinute / endMinute	Number	Minutes since midnight; end must exceed start
type	Enum	L or P
teacherName / classroom	String	Teacher and room/lab
effectiveFrom / effectiveTo	Date	Schedule validity range
scheduleVersion	Number/String	Preserves schedule revisions
active	Boolean	Entry state

classOccurrences (optional materialized records)

Field	Type	Purpose / rule
_id	ObjectId	Occurrence identity
ownerId / semesterId / subjectId	ObjectId	Scope and subject
timetableEntryId	ObjectId	Null or alternative reference for extra class
occurrenceKey	String	Stable key; unique within owner and semester
classDate / startMinute / endMinute	Date/Number	Specific scheduled date/time
type / teacherSnapshot / classroomSnapshot	String	Historical detail snapshot
source	Enum	RECURRING or EXTRA

attendanceRecords

Field	Type	Purpose / rule
_id	ObjectId	Primary key
ownerId / semesterId / subjectId	ObjectId	Ownership and scope
occurrenceKey	String	Identifies a specific class period/date
status	Enum	PRESENT or ABSENT for explicit attendance; cancellation handled as occurrence/exception or explicit status if used
classDate / startMinute / endMinute	Date/Number	Query fields and import matching
type / teacherSnapshot / classroomSnapshot	String	Historical values
blockReason	String	Optional source information; not automatically an attendance adjustment
createdAt / updatedAt	Date	Audit timestamps

classNotes

Field	Type	Purpose / rule
_id	ObjectId	Primary key
ownerId / semesterId / occurrenceKey	ObjectId/String	Owner and class occurrence
topicCovered	String	Optional
assignmentGiven	String	Optional
dueDate	Date	Optional

calendarExceptions / extraClasses

Field	Type	Purpose / rule
_id	ObjectId	Primary key
ownerId / semesterId / date	ObjectId/Date	Scope and date
type	Enum	HOLIDAY, CANCELLED, OVERRIDE or EXTRA class as appropriate
subjectId / occurrenceKey	ObjectId/String	Optional links
details	Object	Reason and one-off schedule details

dlApplications / dlCredits

Field	Type	Purpose / rule
_id	ObjectId	Primary key
ownerId / semesterId	ObjectId	Scope
eventName / startAt / endAt / status	String/Date/Enum	Application data
dlApplicationId / occurrenceKey	ObjectId/String	Credit references
creditStatus / creditedAt	Enum/Date	Separate credit state and audit

importJobs

Field	Type	Purpose / rule
_id	ObjectId	Primary key
ownerId / semesterId / subjectId	ObjectId	Scope
fileType / status	String/Enum	CSV/XLSX; PREVIEWED, COMMITTED, FAILED
rowCounts	Object	Created, updated, skipped, rejected
validationSummary	Array/Object	Row-level messages; avoid retaining uploaded files unnecessarily

archiveCatalog

Field	Type	Purpose / rule
_id	ObjectId	Small live metadata only
ownerId / semesterId	ObjectId	Authorization scope
objectKey	String	Private cloud object reference, not a public URL
archiveStatus	Enum	UPLOADING, VERIFIED, FAILED, DOWNLOAD_ONLY
checksum / recordCounts	String/Object	Integrity metadata
createdAt / verifiedAt	Date	Audit metadata

8.1 Data model cautions

- Do not make a unique index on date + start time + end time alone; legitimate timetable overlaps exist.
- Avoid storing PENDING documents for every class. Infer PENDING when no explicit status exists and no cancellation applies.
- Use a stable occurrence key for each timetable entry and date; do not identify a class by time alone.
- If classOccurrences are materialized, create them only when needed or for exceptions; do not pre-generate the entire semester for every student.
- Keep timetable schedule definitions distinct from attendance snapshots.
- The exact separation between classOccurrences and attendanceRecords can be simplified during implementation, but the logical separation and stable identity must remain.

9. Indexes and query patterns

Indexes must follow actual query filters. Start with the indexes below, then use MongoDB explain plans and measurements to refine them. More indexes are not automatically better: they consume storage and make writes more expensive.

Collection	Index (conceptual order)	Reason
users	Unique email	Login and duplicate-account prevention.
semesters	ownerId + status	List current/completed semester metadata.
subjects	ownerId + semesterId + active	Load selected semester subjects.
timetableEntries	ownerId + semesterId + dayOfWeek + effectiveFrom	Load recurring timetable for a weekday.
timetableEntries	ownerId + semesterId + subjectId + dayOfWeek	Subject/date/time import matching.
attendanceRecords	Unique ownerId + semesterId + occurrenceKey	Prevent duplicate attendance for the same occurrence.
attendanceRecords	ownerId + semesterId + classDate + startMinute	Daily attendance view.
attendanceRecords	ownerId + semesterId + subjectId + classDate	Subject history and calculations.
classNotes	Unique ownerId + semesterId + occurrenceKey	One notes document per occurrence, if that rule is adopted.
classNotes	ownerId + semesterId + dueDate	Upcoming assignments.
calendarExceptions	ownerId + semesterId + date	Find holidays/exceptions for a date.
dlApplications	ownerId + semesterId + status	DL application list and reconciliation.
dlCredits	Unique ownerId + dlApplicationId + occurrenceKey	Prevent duplicate application/occurrence credit.
archiveCatalog	ownerId + semesterId	Authorize and locate the student's downloadable archive.

9.1 Indexing rules

- Put ownerId and semesterId first in compound indexes when application queries always scope by them; verify with real query shapes.
- Use unique indexes to enforce correctness, not only application-side checks.
- Do not rely on indexes as an authorization mechanism; every query still needs server-side owner scoping.
- Use pagination and date-range filters; do not load all attendance records for a semester into the frontend.
- Keep archived detailed records out of live collections, so these indexes primarily serve the active semester.

10. API outline

Base prefix: /api/v1. All academic endpoints require authentication and owner-scoped queries. Route names can be adjusted during implementation as long as the behavior and security contract remain consistent.

Area	Method + route	Purpose
Auth	POST /auth/register	Create account
Auth	POST /auth/login	Authenticate
Auth	POST /auth/logout	End session
Auth	GET /auth/me	Current user profile
Semesters	GET/POST /semesters	List/create semester metadata
Semesters	GET/PUT /semesters/:id	Read/update semester
Subjects	GET/POST /semesters/:id/subjects	List/create subjects
Subjects	PUT/DELETE /subjects/:id	Update/archive subject
Timetable	GET/POST /semesters/:id/timetable	List/create recurring entries
Timetable	PUT/DELETE /timetable/:id	Update/archive an entry or create a future version
Calendar	GET /semesters/:id/calendar?date=YYYY-MM-DD	Compute timetable and exceptions for date
Calendar	POST /semesters/:id/holidays	Create holiday/exception
Calendar	POST /semesters/:id/extra-classes	Create one-off class
Attendance	GET /semesters/:id/attendance?date=YYYY-MM-DD	Daily scheduled classes plus explicit statuses
Attendance	PATCH /occurrences/:id/attendance	Record or correct attendance; handle overlap confirmation
Attendance	GET /semesters/:id/attendance/history	Paginated history
Duty Leave	GET/POST /semesters/:id/dl	List/create DL application
Duty Leave	GET/PATCH /dl/:id	View/update application status
Duty Leave	GET /dl/:id/eligible-periods	List period matches
Duty Leave	POST /dl/:id/reconcile	Reconcile eligible/approved/applied periods
Duty Leave	GET /semesters/:id/dl/summary	Semester DL summary
Import	GET /imports/templates/:type	Download template
Import	POST /imports/preview	Validate and preview file
Import	POST /imports/commit	Commit a confirmed preview
Import	GET /imports/:id	Read import result
Analytics	GET /semesters/:id/dashboard	Current-semester dashboard summary
Analytics	GET /subjects/:id/attendance-summary	Subject calculations
Export	GET /semesters/:id/exports/attendance.csv	Active semester CSV export
Archive	POST /semesters/:id/archive	Begin archive process
Archive	GET /semesters/:id/download	Authorize and stream/download archived file
Query console	POST /query-console/execute	Execute validated, read-only query on active data only

10.1 Response and error conventions

Status	Use
200	Successful read or update
201	Created resource
400	Invalid input, malformed file, or validation error
401	Unauthenticated
403	Authenticated but not authorized
404	Resource not found or not owned by user
409	Duplicate or conflicting operation
413	Upload too large
422	Valid request format but row/business-rule validation failed, if used consistently
429	Rate limit exceeded
500/503	Unexpected server error or required dependency unavailable

11. Authentication, security, and validation

- Hash passwords with bcrypt or Argon2; never store plaintext passwords.
- Derive ownerId from the authenticated principal. Ignore or reject ownerId supplied by the client.
- Scope every MongoDB read, update, delete, import and export by ownerId; verify that referenced subject/semester/occurrence also belongs to the same owner.
- Validate request bodies, path parameters, dates, time ranges, enums, file types, file sizes and query-console syntax.
- Protect uploads from oversized files, malformed spreadsheets, formula injection in exported CSVs, and accidental overwrite.
- Use secure secret configuration via environment variables; commit only .env.example, never real credentials.
- Use least-privilege database credentials. A separate read-only database user for the query console is a possible defense-in-depth improvement, but it does not replace owner-level authorization.
- Do not expose archive object keys as public URLs. Authorize each download and use private object storage with short-lived access or server-side streaming.
- Rate-limit login, import, archive, and query-console endpoints. Limit query execution time and result count.
- Do not log passwords, tokens, sensitive attendance notes, or raw uploaded file contents unnecessarily.

12. Semester archival and downloads

Final decision: cloud archives are retained, but archived detailed data is not reused by the server. Students download their own archive as CSV or Excel and work with it locally.

12.1 Semester-end process

Step	Required behavior
1. Start	Move semester metadata to ARCHIVING; prevent conflicting archive jobs.
2. Prepare	Generate an archive containing attendance, subjects/targets, timetable versions/snapshots, teacher/room details, class notes/assignments, exceptions, and DL applications/credits.
3. Upload	Upload to private cloud object storage under an owner/semester-scoped object key.
4. Verify	Verify checksum, object size, and expected record counts; store verification metadata.
5. Commit archive	Mark archive verified/download-only in archiveCatalog.
6. Cleanup	Only after verification and retention checks, delete detailed records from active MongoDB collections.
7. Retain catalogue	Keep small metadata: owner, semester, object reference, status, checksum, counts and timestamps.
8. Download	Authorize the owner and provide CSV/XLSX downloads. Do not load the archive into MongoDB or use it for server-side analytics/query console.

12.2 Failure safety

- If upload or verification fails, keep detailed MongoDB records and mark archive failed/retryable.
- Never delete source records before successful verification.
- Use an idempotent archive job so retries do not create inconsistent duplicate archives.
- Use private cloud storage, encryption in transit/at rest where supported, and owner authorization on every download.
- Define retention, deletion, and recovery policy before production use. A cloud archive should not be treated as a substitute for all backups unless its durability and recovery design are verified.

12.3 Download formats

CSV is suitable for one table per file. Excel can use multiple worksheets for attendance, timetable, subjects, notes/assignments, exceptions, and Duty Leave. The archive can retain a structured machine-readable package plus generated student-facing exports, provided the application still treats the package as download-only.

12.4 Historical access limitation

After archival, previous semesters are listed as archived and expose download actions. The app does not show archived attendance dashboards, run archived calculators, or execute query-console operations against cloud archive data. Students can perform their own analysis on their downloaded files.

13. Redis-ready architecture

13.1 Version 1 compatibility requirement

Design the application so Redis can be added in Version 2 without rewriting business logic or API routes. Keep cache operations behind a small interface/adapter and provide a no-op implementation for Version 1. MongoDB remains the source of truth.

```
Cache interface (conceptual):
get(key)
set(key, value, ttl)
delete(key)
deleteMany(keys) or invalidateByNamespace(namespace)

V1: NoOpCache (always misses; writes are ignored)
V2: RedisCache (TTL, serialization, connection/retry behavior)
```

13.2 Cache-aside flow

- Service checks the cache for an eligible read.
- On cache hit, return the validated cached result.
- On miss, read MongoDB, then populate cache with a configured TTL.

- After a successful database write, invalidate all affected cache keys.
- If Redis is unavailable, fall back to MongoDB; cache failure must not lose attendance writes.

13.3 Candidate caches

Data	Initial cache decision	Starting TTL
Weekly timetable	Good candidate	5-15 minutes
Subject details/targets	Good candidate	5-15 minutes
Dashboard summary	Good candidate after measurement	1-2 minutes
Today's class list	Candidate; invalidate on timetable/exception changes	1-5 minutes
Attendance writes	Never use cache as source of truth	N/A
Query-console results	Do not cache initially	N/A

13.4 Cache key and invalidation rules

```
dashboard:<ownerId>:<semesterId>
timetable:<ownerId>:<semesterId>:<week>
subject-summary:<ownerId>:<semesterId>:<subjectId>
```

Every key must include owner scope, semester scope and relevant query parameters. Invalidate related entries after attendance changes, imports, timetable changes, exceptions, subject target changes, and DL credit changes. Use configurable TTLs. Never let a cache hit bypass authorization.

13.5 Redis deployment decision

Redis is not a required service in Version 1. First optimize MongoDB indexes and measure response times. Add Redis in Version 2 only for measured, repeated reads. Test with cache disabled and with Redis unavailable; both modes must produce correct results.

14. Version roadmap and delivery plan

14.1 Feature versions

Version / priority	Features	Exit criteria
V1 P0: Core	Auth; current semester; subjects; recurring timetable; class types; separate periods; attendance status; overlap confirmation; calendar exceptions; basic DL tracking; history for active semester.	Student can securely set up semester/timetable, record correct attendance, and calculate accurate current attendance.
V1 P1: Data and insights	Template attendance import, preview/commit, CSV export, dashboard, calculators, restricted query console, archive catalogue/download flow, basic analytics.	Import errors are reviewable, writes are idempotent, exports are authorized, and active-data query console is safe.
V2: Performance and advanced workflow	Redis adapter enabled; measured cache coverage; planner/reminders; enhanced analytics; additional reports and polished archive/download UX.	Caching does not change results; invalidation and failure fallback pass tests.
Later / optional	Advanced forecasting, PDF reports, flexible spreadsheet mapping, richer calendar, advanced aggregation query features, cloud deployment automation.	Only add after core workflows are stable and there is time to validate.

14.2 Seven-day stretch plan

Day	Focus	Priority
1	Initialize frontend/backend, environment config, MongoDB connection, register/login, owner scoping.	Critical
2	Semesters, subjects, timetable CRUD, recurring entries, teacher/room/type, effective dates.	Critical
3	Daily schedule, attendance statuses, overlap confirmation, exceptions and history.	Critical
4	Attendance calculations, basic DL applications, partial-overlap eligibility and credit audit.	Critical
5	Dashboard, attendance UI, calendar and basic responsive layout.	High
6	Template import preview/commit, validation, CSV export, read-only query console basics.	High
7	Security review, tests, archive workflow skeleton, README, demo and bug fixing.	Critical

The seven-day plan is aggressive for a beginner learning React. If time runs short, prioritize secure data isolation, correct attendance behavior, and data integrity. Planner/reminders, advanced analytics, and Redis deployment can follow.

14.3 Implementation approach

- Build and test one module at a time; do not generate the whole application in a single step.
- Use small service functions for calculations and matching rules so they can be tested without the UI.
- Keep API contracts stable and document validation/error behavior.
- Use seed/sample data only for development; do not hardcode student records in production logic.
- Commit frequently and maintain a working README with setup and environment instructions.

15. Testing and acceptance criteria

Area	Acceptance checks
Ownership/security	Student A cannot read/update Student B's subjects, attendance, imports, query results or archive downloads, even when IDs are changed.
Timetable	Consecutive lectures remain separate; practical periods remain separate; overlaps can be saved with warnings; future edits do not alter historical snapshots.
Attendance	PENDING/CANCELLED excluded; PRESENT/ABSENT included; N/A when no eligible periods; retries do not create duplicates.
Overlap marking	Confirmation affects only overlapping PENDING entries; existing statuses are not overwritten; canceling confirmation leaves other entries unchanged.
DL	Partial overlap uses classStart < dlEnd && classEnd > dlStart; full period credit only after approval; no double-credit.
Import	Exact headers validated; selected subject/date/time matching; optional fields enriched from timetable; ambiguity/errors shown; no silent overwrite.
Query console	Write/admin operations rejected; owner filter enforced server-side; sensitive fields hidden; result/time limits enforced.
Archive	Upload/verification failure retains source records; successful archive can be downloaded only by owner; archived data is not loaded/query-processed by app.
Cache readiness	V1 works with no-op cache; V2 cache results match MongoDB results; invalidation after writes; Redis outage falls back safely.
Performance	Paginated reads, indexed query plans and measured response times; do not claim scaling based only on index presence.

15.1 High-value test scenarios

- Two same-subject lectures back-to-back on one date create two distinct attendance identities.
- Two overlapping classes at the same time are both represented independently.
- A class from 13:30 to 14:10 overlaps DL starting at 14:00; a class ending exactly at 14:00 does not.
- An import with blank Type/Name is enriched from timetable when the date/time match is unambiguous.
- An import with missing Attendance or ambiguous time is not committed until corrected.
- A timetable change effective next week leaves last week's attendance snapshots untouched.
- A semester archive upload failure leaves all source data intact.
- A user attempting a query that omits or manipulates owner scope still sees only their own active data.

16. Project structure and implementation instructions

```
student-attendance-tracker/
frontend/
src/
components/
layouts/
pages/
services/ # API client calls
types/ # TypeScript interfaces
utils/ # UI helpers only
App.tsx
main.tsx
backend/
src/
config/ # DB and environment configuration
models/ # Mongoose schemas
routes/ # Express route declarations
controllers/ # HTTP request/response handling
services/ # Business logic and use cases
repositories/ # Data access queries
middleware/ # Auth, validation, rate limits, errors
cache/ # Cache interface + no-op adapter (V1)
utils/ # Shared backend utilities
app.js
server.js
.env.example
docs/
.gitignore
README.md
```

16.1 Layer responsibilities

Layer	Responsibility
Routes	Map HTTP method/path to controller; no business logic.
Controllers	Validate/normalize HTTP input and call services; shape responses.
Services	Attendance rules, overlap behavior, import decisions, calculations, archive orchestration.
Repositories	Owner-scoped MongoDB/Mongoose access and reusable query patterns.
Cache adapter	No-op in V1; Redis implementation in V2; never decides business truth.
Middleware	Authentication, authorization helpers, schema validation, error handling, rate limiting.
Frontend services	Centralized API client and error handling; no direct database access.

16.2 Environment configuration

Use environment variables for MongoDB connection string, authentication secrets, allowed frontend origin, upload limits, archive provider credentials, and optional Redis URL. Keep .env out of version control and commit .env.example with placeholder names only.

16.3 Development instructions

- Do not start coding until requirements are accepted and this plan is treated as the working specification.
- Implement the smallest end-to-end flow first: register/login, create semester/subject, create timetable, mark attendance, view dashboard.
- Add tests for attendance calculations and matching rules before adding UI complexity.
- Do not add Redis as a required dependency in V1; include the interface and no-op implementation.
- Keep archive operations explicit and stateful; never delete live data on an unverified upload.

17. Final decisions, assumptions, and items to define during implementation

17.1 Finalized decisions

- Stack: React + TypeScript + Vite; Node.js + Express; MongoDB + Mongoose.
- Multi-user self-registration; all academic data is owner-scoped.
- Recurring semester timetable with teacher, classroom, and L/P type; edits are effective-dated and do not rewrite history.
- Every class period remains independent, including back-to-back and overlapping classes.
- Attendance statuses are PENDING, PRESENT, ABSENT, CANCELLED with the agreed denominator rules.
- Overlapping attendance marking uses confirmation; only overlapping PENDING classes can be auto-marked ABSENT.
- Duty Leave uses separate credit records; partial interval overlap makes a whole period eligible for review, not automatically credited.
- Attendance import template headers are exactly Date, Time, Type, Attendance, Name, Block Reason; Name means teacher; Block Reason is a college block/disciplinary reason.
- Import matching uses selected subject plus date and time; other columns are optional and may be enriched from the timetable.
- Previous semesters are archived in cloud storage and available for personal CSV/XLSX download only; the application does not query archived data.
- Redis is designed for in Version 1 through a cache abstraction; actual Redis integration is Version 2.
- Restricted read-only query console; no arbitrary writes, administrative commands or cross-user data access.

17.2 Details to specify before implementation

- Exact accepted attendance strings in uploaded files (e.g. Present/Absent, P/A, source-specific labels) and normalization rules.
- Exact time representation in the template: preferred normalized start-end range or a documented college export format.
- Whether a college block changes attendance calculations, and how it differs from an ordinary absence.
- Archive provider, cost/retention policy, supported Excel workbook layout, and whether the archive is a ZIP package plus student-facing exports.
- Session/token strategy and production deployment details.
- Exact aggregation stages permitted in the read-only query console.
- Conflict policy when a holiday overlaps a manually added extra class.

17.3 Out of scope for the first release

- Arbitrary spreadsheet autodetection and flexible column mapping.
- Server-side analytics or query-console access to archived semester data.
- Mandatory Redis deployment in Version 1.
- Automatic creation of all future class occurrences and PENDING attendance documents.
- Advanced forecasting, extensive PDF report generation, and complex reminder delivery until core features are stable.

Final implementation principle: preserve data integrity first. The project should remain correct with MongoDB alone, without Redis, and without access to archived cloud data. Add caching and advanced features only through clear interfaces and tested incremental changes.